Silver-Level TREES

SHSID Competitive Programming Club

Part 1 (from 2024-2025)

Some properties/definitions of trees:

- **A graph is a tree iff it is connected and contains N nodes and N−1 edges**
- **A graph is a tree iff every pair of nodes has exactly one simple path between them**
- **A graph is a tree iff it is connected and does not contain any cycles**

  Other terms: leaf, forests, root, parent, subtree, depth, ancestor, degree, diameter

  notice paths are unique given two endpoints

8VC Venture Cup 2017 - Elimination Round

C. PolandBall and Forest

Problem: given a forest of N nodes and for each node you're given the node that is farthest away from it on its tree. find the number of trees.

Examples

input

```
5
2 1 5 3 3
```

output

```
2
```

input

```
1
1
```

output

```
1
```

8VC Venture Cup 2017 - Elimination Round

C. PolandBall and Forest

Solution idea: notice that within each tree, there is only and exactly one pair of nodes satisfying p[a] = b and p[b] = a ! These two points must constitute a diameter

```cpp
#include <bits/stdc++.h>
using namespace std;

int p[10005];

int main() {
    int n;
    cin >> n;
    for (int i = 1; i <= n; i++) cin >> p[i];

    int ans = 0;
    for (int i = 1; i <= n; i++)
        if (p[p[i]] == i && i <= p[i]) ans++;

    cout << ans << "\n";
    return 0;
}

```

USACO 2018 January Contest, Silver Problem 3. MooTube

Problem: There is a map containing N nodes and N-1 edges, and each of those paths have a value. There is only one connected component in the map. Find the minimum value of an edge over the path from one given node to another given node.

conveniently numbered $1 \ldots N$. However, FJ can't quite figure out how to help his cows find new videos they might like.

FJ wants to create a list of "suggested videos" for every MooTube video. This way, cows will be recommended the videos most relevant to the ones they already watch.

FJ devises a metric of "relevance," which determines, as the name suggests, how relevant two videos are to each other. He picks $N-1$ pairs of videos and manually computes their pairwise relevance. Then, FJ visualizes his videos as a network, where each video is a node and the $N-1$ pairs of videos he manually considered are connected. Conveniently, FJ has picked his $N-1$ pairs so that any video can be reached from any other video along a path of connections in exactly one way. FJ decides that the relevance of any pair of videos should be defined as the minimum relevance of any connection along this path.

Farmer John wants to pick a value K so that next to any given MooTube video, all other videos with relevance at least K to that video will be suggested. However, FJ is worried that too many videos will be suggested to his cows, which could distract them from milk production! Therefore, he wants to carefully set an appropriate value of K. Farmer John would like your help answering a number of questions about the suggested videos for certain values of K.

INPUT FORMAT (file mootube.in):

The first line of input contains N and Q ($1 \leq Q \leq 5000$).

The next $N-1$ lines each describe a pair of videos FJ manually compares. Each line includes three integers p_i, q_i, and r_i ($1 \leq p_i, q_i \leq N, 1 \leq r_i \leq 1,000,000,000$), indicating that videos p_i and q_i are connected with relevance r_i.

The next Q lines describe Farmer John's Q questions. Each line contains two integers, k_i and v_i ($1 \leq k_i \leq 1,000,000,000, 1 \leq v_i \leq N$), indicating that FJ's ith question asks how many videos will be suggested to viewers of video v_i if $K = k_i$.

OUTPUT FORMAT (file mootube.out):

Solution idea: We can use a BFS here! What we do is that we start from the given node and then scan through the tree.

Submitted; Results below show the outcome for each judge test case

*	*	*	*	*	*	*	*	*	*
3.4mb	3.4mb	3.4mb	3.4mb	3.4mb	3.6mb	3.4mb	3.6mb	3.6mb	3.6mb
1 2ms	2 2ms	3 2ms	4 2ms	5 6ms	6 6ms	7 13ms	8 135ms	9 52ms	10 49ms

```
    cin>>n>>q;
    vector<vector<Edge>>edges(n);
    for(int a=1;a<n;a++){
        int x,y,w;
        cin>>x>>y>>w;
        x--;y--;
        edges[x].emplace_back(y,w);
        edges[y].emplace_back(x,w);
    }
    for(int query=0;query<q;query++){
        int threshold,start;
        cin>>threshold>>start;
        start--;
        int ret=0;
        queue<int>q;
        q.push(start);
        vector<bool>seen(n,false);
        seen[start]=true;
        while(!q.empty()){
            int curr=q.front();
            q.pop();
```

```
    freopen("mootube.out","w",stdout);
    int n,q;
    cin>>n>>q;
    vector<vector<Edge>>edges(n);
    for(int a=1;a<n;a++){
        int x,y,w;
        cin>>x>>y>>w;
        x--;y--;
        edges[x].emplace_back(y,w);
        edges[y].emplace_back(x,w);
    }
    for(int query=0;query<q;query++){
        int threshold,start;
        cin>>threshold>>start;
        start--;
        int ret=0;
        queue<int>q;
        q.push(start);
        vector<bool>seen(n,false);
        seen[start]=true;
        while(!q.empty()){
            int curr=q.front();
            q.pop();
            for(const Edge&out:edges[curr]){
                if(!seen[out.d]&&out.w>=threshold){
```

C. Journey

time limit per test: 2 seconds

memory limit per test: 256 megabytes

Problem: There is a tree with n vertices. Find the average path length containing node 1

There are n cities and n - 1 roads in the Seven Kingdoms, each road connects two cities and we can reach any city from any other by the roads.

Theon and Yara Greyjoy are on a horse in the first city, they are starting traveling through the roads. But the weather is foggy, so they can't see where the horse brings them. When the horse reaches a city (including the first one), it goes to one of the cities connected to the current city. But it is a strange horse, it only goes to cities in which they weren't before. In each such city, the horse goes with equal probabilities and it stops when there are no such cities.

Let the length of each road be 1. The journey starts in the city 1. What is the expected length (expected value of length) of their journey? You can read about expected (average) value by the link https://en.wikipedia.org/wiki/Expected_value.

Input

The first line contains a single integer n ($1 \le n \le 100000$) — number of cities.

Then n - 1 lines follow. The i-th line of these lines contains two integers u_i and v_i ($1 \le u_i, v_i \le n, u_i \ne v_i$) — the cities connected by the i-th road.

It is guaranteed that one can reach any city from any other by the roads.

Output

Print a number — the expected length of their journey. The journey starts in the city 1.

Your answer will be considered correct if its absolute or relative error does not exceed 10^{-6}.

Namely: let's assume that your answer is a, and the answer of the jury is b. The checker program will consider your answer correct, if $\frac{|a-b|}{max(1,b)} \le 10^{-6}$.

Examples

input Copy

```
4
1 2
1 3
2 4
```

output Copy

```
1.500000000000000
```

input Copy

```
5
1 2
1 3
3 4
2 5
```

output Copy

```
2.000000000000000
```

Solution idea: Just DFS through the tree; the path only ends once a leaf node is reached (this could be easily seen and proven), so once we see a leaf node we add our stored probability * value to our answer for O(N).

```cpp
#include <bits/stdc++.h>
using namespace std;
const int N = 100005;
double ans;
int n;
vector<int> edges[N];
void dfs(int i,int f,int dep,double prob){
    int moves = 0;
    for(auto x : edges[i]){
        if(x != f){
            moves++;
        }
    }
    if(!moves)ans += dep*prob;
    else{
        double newp = prob/moves;
        for(auto x : edges[i]){
            if(x != f)dfs( i: x, f: i, dep: dep+1, prob: newp);
        }
    }
}
int main() {
    cin >> n;
    for(int i = 1; i < n; i++){
        int a,b;
        cin >> a >> b;
        edges[a].push_back(b);
        edges[b].push_back(a);
    }
    dfs( i: 1, f: 0, dep: 0, prob: 1);
    cout << fixed << setprecision( n: 10) << ans;
    return 0;
}
```

USACO 2019 December Contest, Silver
Problem 3. Milk Visits

```
5 5
HHGHG
1 2
2 3
2 4
1 5
1 4 H
1 4 G
1 3 G
1 3 H
5 5 H
```

Problem: Given n the number of nodes, m the number of tests, then a line of H and G's indicating the type of node from 1 to n, then n-1 edges that connect into a tree. For each of the m testcases, determine if the path between the nodes contains H or G at some point...

USACO 2019 December Contest, Silver
Problem 3. Milk Visits

Say that two cows are in the same connected component if all cows on the path between them are of the same type. This can be done with a DFS with O(N). Then, for two given query points, if they are in different CC, obviously will have both H and G, yes; if same CC, check the type of the start/end point, determines if yes or no

```cpp
#include <bits/stdc++.h>
using namespace std;

vector<int> G[100005];
bool type[100005];
int group[100005];
int n, m;

void dfs(int x, int color, int typ){
    group[x] = color;
    for (int v : G[x]){
        if (type[v] != typ || group[v] != 0) continue;
        dfs(v, color, typ);
    }
}

int main(){
    // freopen("milkvisits.in", "r", stdin);
    // freopen("milkvisits.out", "w", stdout);
    cin >> n >> m;
    for (int i = 1; i <= n; i++){
        char c; cin >> c;
        type[i] = (c == 'H');
    }
    for (int i = 1; i <= n-1; i++){
        int a, b;
        cin >> a >> b;
        G[a].push_back(b);
        G[b].push_back(a);
    }
    int color = 1;
    for (int i = 1; i <= n; i++){
        if (group[i] == 0){
            dfs(i, color, type[i]);
        }
        color++;
    }
```

```cpp

    for (int i = 1; i <= m; i++){
        int a, b; char c;
        cin >> a >> b >> c;
        if (group[a] != group[b]){
            cout << '1';
        } else {
            if (type[a] == (c=='H')){
                cout << '1';
            } else {
                cout << '0';
            }
        }
    }
    cout << endl;

    return 0;
}
```

USACO 2020 December Contest, Silver Problem 1. Cowntagion

Problem: There is a map containing N nodes and N-1 edges that connect them (the map is connected). Every node except one have a value at the start, while the single other node has a value of 1. For each day, one of the nodes could double its value, or one node can decrease its value by 1 and a node connected with it through an edge would increase its value by 1.

Farmer John and his fellow farmers have been working nonstop to control the spread of the terrible bovine disease COWVID-19 across their farms.

Together, they oversee a collection of N farms ($1 \leq N \leq 10^5$), conveniently numbered $1 \ldots N$. The farms are connected by a set of $N-1$ roads such that any farm can be reached from farm 1 by some sequence of roads.

Unfortunately, a cow in farm 1 has just tested positive for COWVID-19. None of the other cows at that farm or at any other farms have the disease yet. However, knowing the contagious nature of the disease, Farmer John anticipates exactly one of the following adverse events on each successive day:

(1) In a single farm, a "superspreader" event causes the number of cows at that farm with COWVID-19 to double; or

(2) A single cow with COWVID-19 moves along a road from one farm to an adjacent farm.

Farmer John is worried about how fast the outbreak might spread. Please help him by determining the minimum possible number of days before it could be the case that at least one cow in every farm has the disease.

Solution idea: We can see that if we want to minimize the amount of days for every farm to have an infected cow, the number of infected farms should increase by each move along an edge. This is since if a farm doesn't have any infected cow, you would have to insert one into it later on, which would be a waste of time. We can see that it is better to increase the number of cows and then spread out the cows to its children, which means we should try to do some doubling first and then spread the cows out.

```cpp
#include<iostream>
#include<vector>
#include<algorithm>
using namespace std;
int n;
int d[100005];
int main(){
    cin>>n;
    int a,b;
    for(int i=0;i<n-1;i++){
        cin>>a>>b;
        a--;
        b--;
        d[a]++;
        d[b]++;
    }
    int ans=n-1;
    for(int i=0;i<n;i++){
        if(d[i]>1||i==0){
            int c=d[i];
            if(i!=0)c--;
            int log=0;
            int pow=1;
            while(pow<c+1){
                log++;
                pow*=2;
            }
            ans+=log;
        }
    }
    cout<<ans;
}
```

Submitted; Results below show the outcome for each judge test case

* 3.4mb 1 2ms	* 3.6mb 2 24ms	* 3.6mb 3 24ms	* 3.6mb 4 24ms	* 3.7mb 5 27ms	* 3.8mb 6 38ms	* 3.8mb 7 38ms	* 3.8mb 8 39ms	* 3.7mb 9 28ms	* 3.7mb 10 35ms	* 3.8mb 11 46ms	* 3.8mb 12 46ms
* 3.8mb 13 46ms	* 3.8mb 14 46ms	* 3.8mb 15 46ms									

CSES Problem Se

Tree Diame

You are given a tree consisting of n nodes.

The *diameter* of a tree is the maximum distance between two nodes. Your task is to determine the diameter of the tree.

Input

The first input line contains an integer n: the number of nodes. The nodes are numbered $1, 2, \ldots, n$.

Then there are $n-1$ lines describing the edges. Each line contains two integers a and b: there is an edge between nodes a and b.

Output

Print one integer: the diameter of the tree.

Given a tree, find its diameter.

Constraints

- $1 \leq n \leq 2 \cdot 10^5$
- $1 \leq a, b \leq n$

Example

Input:

```
5
1 2
1 3
3 4
3 5
```

Output:

```
3
```

Explanation: The diameter corresponds to the path $2 \to 1 \to 3 \to 5$.

CSES Problem Set

Tree Diameter

Solution idea: set an arbitrary point as root u, find the node that's furthest from it, call it v. Then, find the furthest point from v, call it r. vr is a diameter.

Proof. If we can prove that v is the endpoint of one of the diameters, we're done. So, suppose for contradiction that v is not an endpoint and one diameter is s, t.
let x be the junction of u, s, t (the LCA of s, t with root u).
Then, we consider the possible positions of v...

1. **v is attached to xt or xs. Then,**

```
d(s, t) = d(s, x) + d(x, t) <= d(s, x) + d(x, v) = d(s, v),
```

2. **v is attached to ux.**

```
d(u, s) <= d(u, v) <= d(u, x) + d(x, v)
d(u, t) <= d(u, v) <= d(u, x) + d(x, v)

d(s, t)  = d(s, x) + d(x, t)
         = d(u, s) + d(u, t) - 2 d(u, x)
        <= 2 d(x, v)

2 d(s, t) <= d(s, t) + 2 d(x, v)
           = d(s, x) + d(x, v) + d(v, x) + d(x, t)
           = d(v, s) + d(v, t),
```

Both contradict the assumption

```cpp
#include <bits/stdc++.h>
using namespace std;

vector<int> adj[100005];
int dista[100005];

// BFS from src, returns {farthest_node, distance_to_it}
pair<int,int> bfs(int src, int n) {
    fill(dista+1, dista+n+1, -1);
    queue<int> q;
    dista[src] = 0;
    q.push(src);
    int far = src;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : adj[u]) {
            if (dista[v] == -1) {
                dista[v] = dista[u] + 1;
                q.push(v);
                if (dista[v] > dista[far])
                    far = v;
            }
        }
    }
    return {far, dista[far]};
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;
    for (int i = 1; i <= n; i++)
        adj[i].clear();

    for (int i = 0; i < n-1; i++) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }

    // 1) BFS from 1 to find one endpoint of the diameter
    pair<int,int> t1 = bfs(1, n);
    // 2) BFS from that endpoint to find the diameter length
    pair<int,int> t2 = bfs(t1.first, n);

    cout << t2.second << "\n";
    return 0;
}
```

CSES Problem Set

Tree Distances II

You are given a tree consisting of n nodes.
Your task is to determine for each node the sum of the distances from the node to all other nodes.
n within 2e5

Input:

```
5
1 2
1 3
3 4
3 5
```

Output:

```
6 9 5 8 8
```

CSES Problem Set

Tree Distances II

Naive distance finding would take O(N^2)

Let node 1 be the root (arbitrarily)
Precompute the size of each subtree with respect to this root.
Find the sum of distances for node 1, then consider how the sum changes going from 1 to its neighbors etc.
e.g. if edge(1, x): the sum decreases by size(x), increases by n - size(x).

```cpp
#include <bits/stdc++.h>
using namespace std;

int n;
long long suma;
vector<int> G[200005];
int siz[200005];
int ans[200005];

void dfs(int x, int fa, int dep){
    suma += dep;
    siz[x] = 1;
    for (int v : G[x]){
        if (v != fa){
            dfs(v,x,dep+1);
            siz[x] += siz[v];
        }
    }
}

void calcAns(int x, int fa, int s){
    ans[x] = s;
    for (int v : G[x]){
        if (v != fa){
            calcAns(v, x, s-siz[v]+(n-siz[v]));
        }
    }
}

int main(){
    cin >> n;
    for (int i = 1; i <= n-1; i++){
        int a, b;
        cin >> a >> b;
        G[a].push_back(b);
        G[b].push_back(a);
    }
    dfs(1, 1, 0);
    calcAns(1, 1, suma);
    for (int i = 1; i <= n; i++){
        cout << ans[i] << ' ';
    }
    cout << endl;

    return 0;
}
```

Codeforces Round 434 (Div. 1, based on Technocup 2018 Elimination Round 1)

- he will give a performance and show a movie for free at the city y;
- he will drive to some neighboring to y city z;
- he will give a performance and show a movie for free at the city z;
- he will embark the Helicopter and fly away from the city z.

It is known that the wizard doesn't like to use roads, so he agrees to use each road at most once (regardless of direction). In other words, for road between a and b he only can drive once from a to b, or drive once from b to a, or do not use this road at all.

The wizards wants to plan as many episodes as possible without violation the above rules. Help the wizard!

Please note that the wizard can visit the same city multiple times, the restriction is on roads only.

Input

The first line contains two integers n, m ($1 \le n \le 2 \cdot 10^5$, $0 \le m \le 2 \cdot 10^5$) — the number of cities and the number of roads in Berland, respectively.

The roads description follow, one in each line. Each description is a pair of two integers a_i, b_i ($1 \le a_i$, $b_i \le n$, $a_i \ne b_i$), where a_i and b_i are the ids of the cities connected by the i-th road. It is guaranteed that there are no two roads connecting the same pair of cities. Every road is bidirectional. The cities are numbered from 1 to n.

It is possible that the road network in Berland is not connected.

Output

In the first line print w — the maximum possible number of episodes. The next w lines should contain the episodes in format x, y, z — the three integers denoting the ids of the cities in the order of the wizard's visits.

Examples

input

```
4 5
1 2
3 2
2 4
3 4
4 1
```

output

```
2
1 4 2
4 3 2
```

input

```
5 8
5 3
1 2
4 5
5 1
2 5
4 3
1 4
3 2
```

output

```
4
1 4 5
2 3 4
1 5 3
```

Codeforces Round 434 (Div. 1, based on Technocup 2018 Elimination Round 1)

Obviously each CC is independent

For each tree consider from bottom to top:
for each node v if it has x children, if x is even, pair the children nicely; if x is odd, pair one child with v's parent.

With this, exactly floor(m/2) episodes can be made

Silver+ TREES

SHSID Competitive Programming Club

Part 2 (from 2025-2026)